

systemd

Grundlagen

Was ist systemd ?

systemd ist ein System- und Dienstemanager für Linux. Es fungiert als Init-System und ist der erste Prozess (PID 1), der nach dem Start des Linux-Kernels ausgeführt wird. Als Prozess mit der PID 1 übernimmt systemd eine zentrale Rolle im System und startet bzw. verwaltet alle weiteren Prozesse und Dienste. systemd wurde entwickelt, um: (vgl. [SYS1])

- Die Startzeit des Systems zu verkürzen.
- Dienste effizient zu verwalten.
- Abhängigkeiten zwischen Diensten zu steuern.
- Systemressourcen kontrolliert zu verwalten.

Heute ist systemd bei den meisten Linux-Distributionen wie Debian, Ubuntu, Red Hat oder SUSE der Standard.

Historischer Hintergrund (SysVinit)

Vor systemd wurde überwiegend das Init-System SysVinit verwendet. Nachdem der Kernel gestartet war, wurde der Prozess init mit der PID 1 ausgeführt. Dieser startete anschließend weitere Dienste entsprechend eines definierten Runlevels. Die Startskripte befanden sich typischerweise unter /etc/init.d/. (vgl. [SYS1])

Nachteile von SysVinit

- Dienste wurden sequentiell gestartet, wodurch sich längere Bootzeiten ergaben.
- Abhängigkeiten wurden nur indirekt über Startreihenfolgen geregelt.
- Keine integrierte Überwachung oder automatischer Neustart von Diensten.
- Verschiedene separate Werkzeuge für Logging, Zeitsteuerung etc. werden benötigt.

systemd als Problemlöser

systemd ist die zentrale Steuerinstanz moderner Linux-Systeme. Es verwaltet den Systemstart, Dienste, Ressourcen und viele grundlegende Systemfunktionen. Durch paralleles Starten, Abhängigkeitsmanagement und integrierte Überwachung bietet es eine leistungsfähige und einheitliche Lösung zur Systemverwaltung. (vgl. [SYS1])

Vorteile von systemd

- Paralleles Starten von Diensten.
- Deklaratives Abhängigkeitsmanagement.
- Automatische Überwachung und Neustart von Diensten.
- Einheitliche Verwaltungsoberfläche über systemctl.

Der Bootprozess mit systemd

Sobald ein Rechner grundlegend gestartet wird, durchläuft man in Linux vier verschiedene Phasen:

BIOS

Das **Basic Input / Output System (BIOS)** ist die erste Software, die beim Starten eines Computers ausgeführt wird. Es befindet sich auf einem Chip auf der Hauptplatine und ist für die grundlegende Hardwareinitialisierung verantwortlich. Das BIOS führt einen Power-On Self Test (POST) durch, um sicherzustellen, dass die Hardware ordnungsgemäß funktioniert. Es erkennt und konfiguriert die angeschlossenen Geräte wie Festplatten, RAM und Grafikkarten. (vgl. [SYS1])

Bootloader

Der Bootloader ist ein kleines Programm, das nach dem BIOS ausgeführt wird. Der Bootloader selbst befindet sich im **Master Boot Record (MBR)**. Er ist dafür verantwortlich, das Betriebssystem zu laden und zu starten. Der MBR kann dabei entweder auf einer Festplatte, auf einem USB-Stick oder einer anderen bootfähigen Speicherkomponente gespeichert sein. Es gibt verschiedene Bootloader, die in Linux verwendet werden können, wie zum Beispiel GRUB (GRand Unified Bootloader), GRUB2 oder LILO. Der Bootloader bietet in der Regel ein Menü, in dem der Benutzer das gewünschte Betriebssystem auswählen kann, falls mehrere Betriebssysteme installiert sind. (vgl. [SYS1])

Initialisierung Linux Kernel

Nachdem der Bootloader das Betriebssystem ausgewählt und geladen hat, wird der Linux-Kernel gestartet. Der Kernel ist der zentrale Bestandteil des Betriebssystems und verwaltet die Hardware-Ressourcen sowie die Kommunikation zwischen Software und Hardware. Während der Initialisierung des Kernels werden verschiedene Treiber geladen, um die Hardware zu unterstützen. (vgl. [SYS1])

systemd

Nach der Initialisierung des Kernels wird `systemd` als erstes Programm mit der PID 1 gestartet. `systemd` übernimmt die Kontrolle über den Startprozess und ist für das Starten und Verwalten aller weiteren Prozesse und Dienste verantwortlich. Es startet die benötigten Dienste, um das System funktionsfähig zu machen, und sorgt dafür, dass die Abhängigkeiten zwischen den Diensten korrekt gehandhabt werden. (vgl. [SYS1])

Units

Damit `systemd` die verschiedenen Dienste, die auf einem Linux-System laufen, verwalten kann, werden sogenannte **Units** verwendet. Eine Unit ist eine Konfigurationseinheit, die beschreibt, wie ein Dienst gestartet, gestoppt und verwaltet wird. Es gibt verschiedene Arten von Units, darunter: (vgl. [SYS2])

Service Units

Service Units sind die häufigste Art von Units und beschreiben, wie ein Dienst gestartet und verwaltet wird. Sie enthalten Informationen über den ausführbaren Befehl, die Abhängigkeiten zu anderen Diensten, die Startreihenfolge und andere Konfigurationsoptionen. Service Units haben die Dateiendung `.service`. (vgl. [SYS2])

Timer Units

Timer Units werden verwendet, um Dienste zu bestimmten Zeiten oder in regelmäßigen Abständen zu starten. Sie können beispielsweise so konfiguriert werden, dass ein Dienst jeden Tag um Mitternacht oder alle 15 Minuten gestartet wird. Timer Units haben die Dateiendung `.timer` und übernehmen Aufgaben, welche ursprünglich mit Hilfe von Cronjobs erledigt wurden. (vgl. [SYS2])

Path Units

Mit Hilfe von `.path` Units können Dienste gestartet werden, wenn bestimmte Dateien oder Verzeichnisse geändert werden. Sie überwachen also die Dateisystemereignisse und können beispielsweise einen Dienst starten, wenn eine neue Datei in einem bestimmten Verzeichnis erstellt wird. (vgl. [SYS2])

Mount Units

Mount Units beschreiben, wie Dateisysteme gemountet werden sollen. Sie enthalten Informationen über die zu mountende Partition, den Mountpunkt und die Mountoptionen. Mount Units haben die Dateieindung `.mount` . (vgl. [SYS2])

Automount Units

Mit Hilfe der Units mit Dateieindung `.automount` können Dateisysteme automatisch gemountet werden, wenn auf sie zugegriffen wird. Sie arbeiten eng mit Mount Units zusammen und ermöglichen es, dass Dateisysteme erst dann gemountet werden, wenn sie tatsächlich benötigt werden. (vgl. [SYS2])

Socket Units

Sockets sind grundlegend Kommunikationsendpunkte, welche von Diensten genutzt werden, um miteinander zu kommunizieren. Socket Units beschreiben, wie Sockets erstellt und verwaltet werden sollen. Sie enthalten Informationen über die Art des Sockets (z.B. TCP, UDP oder Unix-Socket) und die zugehörigen Dienste. Socket Units haben die Dateieindung `.socket` . (vgl. [SYS2])

Device Units

`.device` Units dienen für die Überwachung von Geräten. Sofern ein Gerät angeschlossen wird, kann zum Beispiel ein Dienst gestartet werden, welcher Funktionalität für das Gerät bereitstellt. (vgl. [SYS2])

Swap Units

Swap Units beschreiben, wie Swap Speicher verwaltet werden soll. Sie enthalten Informationen über die zu verwendende Swap Partition oder Swap Datei und die entsprechenden Optionen. Swap Units haben die Dateieindung `.swap` . (vgl. [SYS2])

Target Units

Target Units mit der Dateieindung `.target` dienen als Zustandsdefinition für das System. Sie werden verwendet, um verschiedene Systemzustände zu definieren, welche erreicht bzw. angesteuert werden sollten. Sie stehen in enger Verbindung mit Service Units, da diese wiederum gestartet werden können, wenn ein bestimmtes Target erreicht worden ist. (vgl. [SYS2])

Slice Units

Slice Units dienen zur Verwaltung von Ressourcen. Sie ermöglichen es, Ressourcen wie CPU, Speicher oder I/O-Bandbreite für verschiedene Dienste oder Gruppen von Diensten zu reservieren und zu begrenzen. Slice Units haben die Dateieindung `.slice` . (vgl. [SYS2])

Scope Units

Scope Units werden verwendet, um Prozesse zu verwalten, die nicht von `systemd` gestartet wurden. Sie ermöglichen es, externe Prozesse in die Verwaltung von `systemd` einzubeziehen und Ressourcen für diese Prozesse zu reservieren. Scope Units haben die Dateieindung `.scope` . (vgl. [SYS2])

Unit Dateien

Struktur

Speicherort von Unit Dateien

Die meisten Unit Dateien befinden sich in einem der folgenden Verzeichnisse:

- `/etc/systemd/system/` :
Hier befinden sich die benutzerdefinierten Unit Dateien, die von Administratoren erstellt oder angepasst wurden. Diese Dateien haben Vorrang vor den Standard-Unit Dateien. (vgl. [SYS1])

- `/lib/systemd/system/` :
Hier befinden sich die Unit Dateien, die von der Distribution bereitgestellt werden. Diese Dateien sollten nicht direkt bearbeitet werden, da sie bei Updates überschrieben werden können. Stattdessen sollten Anpassungen in `/etc/systemd/system/` vorgenommen werden, um diese Dateien zu überschreiben oder zu erweitern. (vgl. [SYS1])
- `/run/systemd/system/` :
Hier befinden sich temporäre Unit Dateien, die während der Laufzeit erstellt werden. Diese Dateien werden normalerweise von `systemd` selbst oder von anderen Diensten generiert und sollten nicht manuell bearbeitet werden. (vgl. [SYS1])

Abschnitte der Unit Dateien

Unit Dateien bestehen aus verschiedenen Abschnitten, die jeweils spezifische Informationen und Anweisungen enthalten. Die wichtigsten Abschnitte sind:

- `[Unit]` :
Dieser Abschnitt enthält allgemeine Informationen über die Unit, wie z.B. eine Beschreibung, Abhängigkeiten zu anderen Units und die Startreihenfolge. (vgl. [SYS1])
- `[<Typ>]` :
Dieser Abschnitt enthält spezifische Anweisungen für den jeweiligen Unit Typ, z.B. `[Service]` für Service Units oder `[Timer]` für Timer Units. Hier werden die eigentlichen Konfigurationsoptionen definiert, wie z.B. der auszuführende Befehl, die Startbedingungen oder die Zeitsteuerung. (vgl. [SYS1])
- `[Install]` :
Hierbei handelt es sich um den letzten Abschnitt der Datei, welcher auch optional ist. Er wird hauptsächlich genutzt, damit das Verhalten bei Aktivierung bzw. Deaktivierung der Unit definiert werden kann. (vgl. [SYS1])

Beispiel für eine Unit Datei

```
[Unit]
Description=Mein SuperDienst
After=network.service

[Service]
Type=simple
WorkingDirectory=/home/hans/web
User=hans
Group=hans
ExecStart=python3 -m http.server 8888
ExecStartPost=/bin/echo "Meinen Dienst gestartet gestartet. PID:" $MAINPID
ExecStopPost=/bin/echo "SuperDienst gekillt"

[Install]
WantedBy=multi-user.target
```

INI

Erläuterungen zur Datei:

- `Description` : Hier wird eine kurze Beschreibung der Unit angegeben.
- `After` : Hier wird angegeben, dass die Unit erst gestartet werden soll, nachdem die `network.service` gestartet wurde.
- `Type` : Hier wird der Typ der Service Unit angegeben. In diesem Fall handelt es sich um eine einfache Service Unit, die direkt gestartet wird.
- `WorkingDirectory` : Hier wird das Arbeitsverzeichnis angegeben, in dem der Dienst ausgeführt werden soll.
- `User` und `Group` : Hier wird der Benutzer und die Gruppe angegeben, unter der der Dienst ausgeführt werden soll.

- **ExecStart** : Hier wird der Befehl angegeben, der ausgeführt werden soll, um den Dienst zu starten. In diesem Fall wird ein einfacher HTTP-Server mit Python gestartet, der auf Port 8888 lauscht.
- **ExecStartPost** : Der Befehl, welcher nach dem Starten des Dienstes ausgeführt wird. In diesem Fall wird eine Nachricht mit der PID des gestarteten Dienstes ausgegeben.
- **ExecStopPost** : Der Befehl, welcher nach dem Stoppen des Dienstes ausgeführt wird. In diesem Fall wird eine Nachricht ausgegeben, dass der Dienst gestoppt wurde.
- **WantedBy** : Hier wird angegeben, dass die Unit aktiviert werden soll, wenn das `multi-user.target` erreicht wird. Das bedeutet, dass der Dienst automatisch gestartet wird, wenn das System in den Mehrbenutzermodus wechselt.

Abhängigkeiten und Reihenfolgen innerhalb der Unit Datei

Grundlegend gibt es zwei verschiedene Arten von Abhängigkeiten und Reihenfolgen, welche innerhalb einer Unit Datei definiert werden können:

Abhängigkeiten einer Unit

Abhängigkeiten werden innerhalb von Units genutzt, da sie es ermöglichen, die Startreihenfolge von Units zu definieren. Für manche Dateien werden zunächst manche andere Units benötigt, bevor die eigentliche Unit überhaupt ausgeführt werden kann. Diese Art von Abhängigkeiten kann mit Hilfe von `Wants` und `Requires` definiert werden: (vgl. [SYS1])

- **Wants** :
Hierbei handelt es sich um eine schwache Abhängigkeit. Wenn die Unit, welche mit `Wants` definiert wurde, nicht gestartet werden kann, wird die eigentliche Unit trotzdem gestartet. Es wird lediglich eine Warnung ausgegeben. Benötigt Unit A zum Beispiel Unit B, so wird beim Start von Unit A auch B gestartet. Sollte B nicht gestartet werden, kann A trotzdem ausgeführt werden. (vgl. [SYS1])
- **Requires** :
`Requires` stellt eine starke Abhängigkeit dar. Wenn die Unit, welche mit `Requires` definiert wurde, nicht gestartet werden kann, wird die eigentliche Unit ebenfalls nicht gestartet. Es wird eine Fehlermeldung ausgegeben. Benötigt Unit A zum Beispiel Unit B, so wird beim Start von Unit A auch B gestartet. Sollte B nicht gestartet werden, kann A ebenfalls nicht ausgeführt werden. (vgl. [SYS1])

Reihenfolgen von Units

Damit ein Linux System in Bezug auf die Funktionsfähigkeit ordentlich starten kann, ist die Startreihenfolge entsprechender Dienste extremst relevant. Ohne Spezifizierung ist `systemd` in der Lage eine Gruppe von Units gleichzeitig auszuführen. Möchte man allerdings Einfluss auf die Reihenfolge nehmen, so gibt es die beiden Schlüsselwörter `Before` und `After` : (vgl. [SYS1])

- **Before** :
`Before` gibt die Reihenfolge an, vor welcher Unit die eigentliche Unit ausgeführt werden soll. Hat Unit A zum Beispiel `Before=unit B`, so wird Unit A vor Unit B gestartet. (vgl. [SYS1])
- **After** :
`After` gibt die Reihenfolge an, nach welcher Unit die eigentliche Unit ausgeführt werden soll. Hat Unit A zum Beispiel `After=unit B`, so wird Unit A nach Unit B gestartet. (vgl. [SYS1])

Wesentliche Befehle

Systeminformationen

Möchte man Informationen über `systemd` und die laufenden Dienste erhalten, so gibt es hierfür verschiedenste Möglichkeiten.

Anzeige laufender / aktiver Dienste

```
systemctl list-units
```

Mit Hilfe des Kommandos können alle geladenen und aktiven Units eines Systems angezeigt werden. Innerhalb der Ausgabe erhält man als Informationen:

- **UNIT :**
Der entsprechende Name der Unit.
- **LOAD :**
Hier wird angezeigt, ob die Unit geladen ist oder nicht.
- **ACTIVE :**
Hier wird angezeigt, ob die Unit aktiv ist oder nicht.
- **SUB :**
Hier wird der genaue Zustand der Unit angezeigt.
- **DESCRIPTION :**
Hier wird eine kurze Beschreibung der Unit angezeigt.

Mögliche weitere Optionen für den Befehl sind:

- **--type=TYPE :**
Hiermit können die angezeigten Units auf einen bestimmten Typ gefiltert werden, z.B. `--type=service` für Service Units oder `--type=timer` für Timer Units.
- **--state=STATE :**
Hiermit können die angezeigten Units auf einen bestimmten Zustand gefiltert werden, z.B. `--state=active` für aktive Units oder `--state=failed` für fehlgeschlagene Units.
- **--all :**
Hiermit werden alle Units angezeigt, unabhängig von ihrem Zustand oder Typ. Standardmäßig werden nur aktive Units angezeigt.

Anzeige aller installierten Dienste

```
systemctl list-unit-files
```

TERMINAL

Mit Hilfe des Kommandos können alle installierten Units eines Systems angezeigt werden. Hier wird folglich auf alle Speicherorte von Unit Dateien geschaut und die darin enthaltenen Units aufgelistet.

Anzeige von Abhängigkeiten der einzelnen Units

```
systemctl list-dependencies [UNIT]
```

TERMINAL

Ohne die entsprechende Angabe einer Unit werden die Abhängigkeiten aller Units angezeigt. Mit der Angabe einer Unit können die Abhängigkeiten einer bestimmten Unit angezeigt werden.

Systemzustand ändern

```
systemctl halt | poweroff | reboot | suspend ...
```

TERMINAL

Mit Hilfe des Kommandos können verschiedene Systemzustände erreicht werden, wie z.B. das Herunterfahren (`halt` oder `poweroff`), der Neustart (`reboot`) oder das Versetzen in den Ruhezustand (`suspend`).

Bootvorgang analysieren

```
systemd-analyze
```

Mit Hilfe des Kommandos können Informationen über die Dauer des Bootvorgangs und die einzelnen Schritte des Startprozesses angezeigt werden. Es gibt verschiedene Optionen, um die Ausgabe zu filtern oder zu formatieren, z.B. `systemd-analyze blame` für eine detaillierte Aufschlüsselung der Startzeiten der einzelnen Dienste.

Vewaltung von Units

Statusprüfung

```
systemctl status UNIT
```

TERMINAL

Anzeige von Dienstinformationen

```
systemctl show UNIT
```

TERMINAL

Starten eines Dienstes

```
systemctl start UNIT
```

TERMINAL

Stoppen eines Dienstes

```
systemctl stop UNIT
```

TERMINAL

Neustarten eines Dienstes

```
systemctl restart UNIT
```

TERMINAL

Neuladen eines Dienstes

```
systemctl reload UNIT
```

TERMINAL

Aktivieren eines Dienstes beim Booten

```
systemctl enable UNIT
```

TERMINAL

Deaktivieren eines Dienstes beim Booten

```
systemctl disable UNIT
```

TERMINAL

Dienste analysieren

Allgemeines

Da es immer wieder zu entsprechenden Problemen kommen kann, ist es wichtig entsprechende Protokolle zu analysieren, um so die Ursachen für die Probleme auch herausfinden zu können. In Linux gibt es hierfür das Kommando `journalctl`, womit Protokolle von `systemd` analysiert werden können! (vgl. [SYS3])

Grundlegend erlaubt `journalctl` den Zugriff auf Ssystemprotokolle aus `systemd-journald` Dateien. Bei diesen Systemprotokollen handelt es sich um Binärdateien, in welchen Meldungen und Ereignisse während des Betriebs eines Systems gespeichert werden. Somit wird eine Möglichkeit geschaffen, dass entsprechende Protokolldaten in Echtzeit durchsucht, gefilter und eingesehen werden können. (vgl. [SYS3])

Grundlegende Konfiguration

Da mit Protokollen natürlich auch eine entsprechende Datenmenge verbunden ist, bietet `journalctl` die Möglichkeit, dass Speicherplatz verwaltet und auch wieder freigegeben werden kann. (vgl. [SYS3])

Speicherplatz verwalten

Bevor der eigentliche Speicherplatz angepasst wird, sollte man zunächst einmal die aktuelle Speicherplatzbelegung überprüfen. Dies kann mit dem folgenden Befehl erfolgen:

```
journalctl --disk-usage
```

TERMINAL

Als Ergebnis erhält man zum Beispiel:

```
Archived and active journals take up 24M in the file system.
```

Löschen von alten Protokolleinträgen

Stellt man fest, dass aktuell zum Beispiel der Speicher zu voll ist, so können alte Protokolleinträge gelöscht werden.

Hierfür gibt es verschiedene Möglichkeiten:

```
journalctl --vacuum-size=SIZE
```

TERMINAL

Über den Zusatz `--vacuum-size=SIZE` können Protokolleinträge gelöscht werden, bis die angegebene Größe erreicht ist. Zum Beispiel könnte man mit `--vacuum-size=100M` alle Protokolleinträge löschen, bis die verbleibende Größe der Protokolleinträge 100 MB beträgt.

Im Gegensatz dazu kann auch nachfolgender Befehl verwendet werden:

```
journalctl --vacuum-time=TIME
```

TERMINAL

Hiermit können Protokolleinträge gelöscht werden, die älter als die angegebene Zeitangabe sind. Zum Beispiel könnte man mit `--vacuum-time=2weeks` alle Protokolleinträge löschen, die älter als 2 Wochen sind. Weitere mögliche Zeitangaben sind zum Beispiel `1month` für 1 Monat oder `3days` für 3 Tage.

Anpassen des Speichers

Um zum Beispiel die maximale Größe von Protokolleinträgen oder auch die Persistenz von Protokolleinträgen anzupassen, müssen entsprechende Einstellungen unter `/etc/systemd/journald.conf` vorgenommen werden:

```
[Journal]
Storage=persistent
#ForwardToSyslog=yes
SystemMaxUse=200M
MaxLevelStore=debug
```

TERMINAL

- **Storage=persistent :**

Hiermit wird festgelegt, dass die Protokolleinträge persistent gespeichert werden sollen. Normalerweise speichert `journald` Protokolleinträge im Arbeitsspeicher unter `/run/log/journal/`. Mit der `persistent` Einstellung hingegen, werden die Einträge unter `/var/log/journal/` gespeichert. Hierdurch gehen die Protokolleinträge auch nach einem Neustart oder nach einem Systemabsturz nicht verloren.

- **ForwardToSyslog=yes :**

Hiermit wird festgelegt, dass die Protokolleinträge auch an den Syslog-Dienst weitergeleitet werden sollen. Hierdurch können die Protokolleinträge auch von anderen Tools wie zum Beispiel `rsyslog` oder `syslog-ng` verarbeitet und gespeichert werden. Ein Auskommentieren der Option führt dazu, dass Protokolleinträge nicht doppelt abgelegt werden!

- `SystemMaxUse=200M` :
Hiermit wird die maximale Größe von Protokolleinträgen auf 200 MB festgelegt. Sobald diese Größe erreicht ist, werden alte Protokolleinträge gelöscht, um Platz für neue Einträge zu schaffen.
- `MaxLevelStore=debug` :
Hiermit wird festgelegt, dass alle Protokolleinträge bis zum Level `debug` gespeichert werden sollen. Es gibt natürlich hierbei noch eine Reihe weiterer Levels:
 - `emerg` :
System wäre unbrauchbar.
 - `alert` :
Es muss sofort gehandelt werden.
 - `crit` :
Schwerwiegender Fehler.
 - `err` :
Fehler.
 - `warning` :
Warnung.
 - `notice` :
Normal, aber bedeutend.
 - `info` :
Informativ.
 - `debug` :
Debugging-Informationen, also alles wird protokolliert.

Persistenz schaffen

Damit Logs wirklich persistent auf einem System abgelegt werden können, muss die nachfolgende Befehlskette ausgeführt werden:

```
mkdir -p /var/log/journal  
  
systemd-tmpfiles --create --prefix /var/log/journal  
  
systemctl restart systemd-journald
```

TERMINAL

- `mkdir -p /var/log/journal` :
Hiermit wird das Verzeichnis `/var/log/journal/` erstellt, falls es noch nicht existiert. Hier werden die Protokolleinträge gespeichert, wenn die `Storage=persistent` Einstellung in der `journald.conf` Datei gesetzt ist.
- `systemd-tmpfiles --create --prefix /var/log/journal` :
Hiermit werden die notwendigen temporären Dateien und Verzeichnisse für die Protokollierung erstellt. Dies ist notwendig, damit `journald` ordnungsgemäß funktioniert und die Protokolleinträge gespeichert werden können. Grundsätzlich betrifft das vor allem den Eigentümer, entsprechende Rechte und die Struktur wie Logdateien organisiert werden.
- `systemctl restart systemd-journald` :
Hiermit wird der `systemd-journald` Dienst neu gestartet, damit die Änderungen in der `journald.conf` Datei wirksam werden und die Protokolleinträge nun persistent gespeichert werden können.

Die Arbeit mit `journalctl`

Anzeige aller Protokolleinträge

```
journalctl
```

TERMINAL

Dieser Befehl wird gerne kombiniert mit den nachfolgenden Optionen:

```
journalctl -xe
```

TERMINAL

- `-x` : + Hiermit werden zusätzliche Erklärungen zu den Protokolleinträgen angezeigt, um so die Fehlersuche zu erleichtern.
- `-e` : + Hiermit wird direkt zum Ende der Protokolleinträge gescrollt, um so die neuesten Einträge anzuzeigen.

Logs live verfolgen

```
journalctl -f
```

TERMINAL

Der Parameter `-f` steht für `follow` und ermöglicht es, die Protokolleinträge in Echtzeit zu verfolgen. Sobald neue Einträge hinzukommen, werden diese automatisch angezeigt. Dieses Feature ist sehr nützlich beim Starten bzw. Debuggen eines Services!

Logs eines bestimmten Services monitoren

```
journalctl -u UNIT
```

TERMINAL

`-u` ist die Abkürzung für `Unit` und erlaubt, dass nur Protokolleinträge eines ganz bestimmten Services angezeigt werden.

Weitere wichtige Parameter

- `-p PRIORITY` :
Hiermit können Protokolleinträge nach ihrer Priorität gefiltert werden. Mögliche Prioritäten sind zum Beispiel `emerg`, `alert`, `crit`, `err`, `warning`, `notice`, `info` und `debug`.
- `--since TIME` und `--until TIME` :
Mit diesen beiden Parametern können Protokolleinträge aus einem bestimmten Zeitraum angezeigt werden. Zum Beispiel könnte man mit `--since "2024-01-01"` alle Protokolleinträge anzeigen, die seit dem 1. Januar 2024 erstellt wurden. Mit `--until "2024-01-31"` könnten alle Protokolleinträge angezeigt werden, die bis zum 31. Januar 2024 erstellt wurden. Es können auch relative Zeitangaben verwendet werden, wie zum Beispiel `--since "2 hours ago"` für alle Protokolleinträge der letzten 2 Stunden.
- `-b` :
Hiermit können Protokolleinträge seit dem letzten Systemstart angezeigt werden. Dies ist besonders nützlich, um Probleme zu identifizieren, die während des aktuellen Betriebs aufgetreten sind.

Timer Units

Cron

Wenn es um zeitgesteuerte Aufgaben geht, denken viele sofort an einen Cron. Hierbei handelt es sich grundlegend um einen Dienst unter Linux, der in der Lage ist, Befehle und Skripte automatisch zu ganz bestimmten Zeiten auszuführen. Somit können mit einem Cron beispielsweise regelmäßig Backups erstellt, Updates geprüft bzw. durchgeführt, Skripte regelmäßig gestartet oder auch E-Mails verschickt werden. (vgl. [SYS5])

Um Cron überhaupt verwenden zu können, muss der entsprechende Dienst auf einem Linux System installiert werden:

```
apt install cron
```

TERMINAL

Nach der Installation können die entsprechenden Jobs verwaltet werden:

Wesentliche Befehle

Anzeige der Jobs für einen Benutzer

```
crontab -l
```

TERMINAL

Bearbeitung der Jobs für einen Benutzer

```
crontab -e
```

TERMINAL

Crons eines anderen Benutzers anzeigen

```
crontab -u USERNAME -l
```

TERMINAL

Format der Cron Jobs

Ein Cron Job besteht aus sechs Feldern, welche durch Leerzeichen voneinander getrennt sind. Die ersten fünf Felder geben an, wann der Job ausgeführt werden soll, während das sechste Feld den Befehl oder das Skript enthält, welches ausgeführt werden soll: (vgl. [SYS5])

```
* * * * * Befehl
| | | | |
| | | | |  Wochentag (0-7) 0 bzw. 7 = Sonntag
| | | | |  Monat (1-12)
| | | | |  Tag im Monat (1-31)
| | | | |  Stunde (0-23)
| | | | |  Minute (0-59)
```

TERMINAL

Ein Beispiel für eine Vollsicherung

Für einen bestimmten Ordner z. B. `/root/ansible` soll eine entsprechende Vollsicherung eingerichtet werden, welche zum Testen gerade im Moment alle 2 Minuten ausgeführt wird. Später wird die Vollsicherung so eingestellt, dass sie wöchentlich jeweils am Freitag Abend stattfinden sollte! Hierfür wird das nachfolgende Coding verwendet:

```
#!/bin/bash

if [ $# -ne 2 ]; then
    echo "Usage: $0 SOURCE BACKUP_DIR"
    exit 1
fi

DATE=$(date +%Y-%m-%d-%H%M%S)
SOURCE="$1"
BACKUP_DIR="$2"

mkdir -p "$BACKUP_DIR"

echo "Make backup $DATE"
echo "===== "

tar -cvzpf "$BACKUP_DIR/backup-$DATE.tar.gz" "$SOURCE"

echo "===== "
echo "Backup finished"
```

TERMINAL

Das Skript wird unter `/root/scripts/backup.sh` abgelegt und muss im Anschluss auch ausführbar gemacht werden:

```
chmod 740 backup.sh
```

TERMINAL

Für den Benutzer `root` wird jetzt ein entsprechender Job erstellt, der alle 2 Minuten ausgeführt wird:

TERMINAL

```
crontab -e
```

TERMINAL

```
* /2 * * * * /root/scripts/backup.sh /root/ansible /tmp/backup
```

Um nicht ständig im entsprechenden Verzeichnis nachzuschauen, ob die Sicherung auch tatsächlich angelegt wurde, können die Protokolleinträge mit `journalctl` verfolgt werden:

TERMINAL

```
journalctl -u cron
```

Das Problem hierbei ist, dass die Protokolleinträge sämtliche Crons enthalten, welche auf dem System ausgeführt werden. Um dieses Problem zu beheben, könnte direkt im Cron selbst festgelegt werden, dass die Protokolleinträge in eine eigene Datei geschrieben werden. Hierfür muss die folgende Zeile in den Cron Job eingefügt werden:

TERMINAL

```
* /2 * * * * /root/scripts/backup.sh /root/ansible /tmp/backup >> /var/log/backup.log 2>&1
```

Mit diesem Zusatz wird die Ausgabe des Cron Jobs in die Datei `/var/log/backup.log` umgeleitet. Hierdurch können die Protokolleinträge für diesen Cron Job separat von anderen Cron Jobs verfolgt werden. Es ist wichtig zu beachten, dass die Datei `/var/log/backup.log` existieren muss und die entsprechenden Berechtigungen haben muss, damit der Cron Job darin schreiben kann:

TERMINAL

```
touch /var/log/backup.log
chmod 664 /var/log/backup.log
```

`2>&1` sorgt dafür, dass sowohl die Standardausgabe als auch die Standardfehlerausgabe in die Datei umgeleitet werden. Somit werden alle Ausgaben des Cron Jobs in der Datei protokolliert, was die Fehlersuche erleichtert.

Möchte man nun zum Beispiel die Vollsicherung immer an einem Sonntag um 23:00 Uhr ausführen, so kann der entsprechende Cron angepasst werden:

TERMINAL

```
0 23 * * 0 /root/scripts/backup.sh /root/ansible /tmp/backup >> /var/log/backup.log 2>&1
```

Ein Beispiel für eine inkrementelle Sicherung

Die inkrementelle Sicherung speichert immer nur die Dateien ab, welche seit der letzten Vollsicherung geändert worden sind. Es kann folglich das nachfolgenden Skript z. B. `backup.sh` verwendet werden:

TERMINAL

```
#!/bin/bash

SOURCE="$1" (1)
DESTINATION="$2" (2)
SNAPSHOT="$DESTINATION/timestamp.dat" (3)
DATE=$(date +%Y-%m-%d-%H%M%S) (4)

mkdir -p "$DESTINATION" (5)

echo "Starting backup..."

tar -czpf "$DESTINATION/backup-$DATE.tar.gz" -g "$SNAPSHOT" "$SOURCE" (6)

echo "Backup created: $DESTINATION/backup-$DATE.tar.gz"
echo "====="
```

Das Skript muss ähnlich zur Vollsicherung ebenfalls wieder ausführbar gemacht werden:

```
chmod 740 backup.sh
```

Bei einer inkrementellen Sicherung macht es Sinn, dass die Sicherung regelmäßig, z. B. jeden Tag um 23:00 Uhr ausgeführt wird. Dies kann mit nachfolgendem Cron Job erreicht werden:

```
0 23 * * * /root/scripts/backup.sh /root/ansible /tmp/backup >> /var/log/backup.log 2>&1
```

Vorteile der Timer Units

Die Timer Units sind nichts anderes als Dienste, welche sich ähnlich Crons verhalten. Der Unterschied liegt jedoch darin, dass sämtliche Vorteile von `systemd` auch hier greifen: (vgl. [SYS4])

- Abhängigkeiten können definiert werden.
- Logs werden via `journalctl` geschrieben.
- Einfache Aktivierung bzw. Deaktivierung.
- Auslösung erfolgt z. B. durch bestimmte Events im System.

Umschreiben von Cron zu Timer Units

Im vorausgegangenen Abschnitt wurde bereits der Cron für die Ausführung von zeitgesteuerten Backups (voll / inkrementell) dargestellt. Entsprechend soll nun die gleiche Aufgabe mit Timer Units umgesetzt werden. Hierfür wird zunächst eine Service Unit erstellt, welche die eigentliche Aufgabe (Backup) enthält. Anschließend wird eine Timer Unit erstellt, welche die Service Unit zu den gewünschten Zeiten ausführt. (vgl. [SYS4])

Erstellung der Service Unit

Die Service Unit wird unter `/etc/systemd/system/backup-full.service` mit nachfolgendem Inhalt erzeugt:

```
[Unit]
Description=Vollsicherung von /root/ansible

[Service]
Type=oneshot
ExecStart=/root/scripts/backup.sh /root/ansible /tmp/backup
```

Es handelt sich hierbei um einen ganz klassischen Service, der allerdings als Typ `oneshot` definiert wird. Hierdurch wird festgelegt, dass der Service nur einmal ausgeführt wird und sich danach automatisch beendet. In diesem Fall wird das Skript `backup.sh` mit den entsprechenden Parametern aufgerufen, um die Vollsicherung durchzuführen.

Erstellung der Timer Unit

Damit der Service ordentlich getestet werden kann, wird zunächst eine Timer Unit erstellt, welche den Service alle 2 Minuten ausführt. Hierfür wird die Datei `/etc/systemd/system/backup-full.timer` mit folgendem Inhalt angelegt:

```
[Unit]
Description=Timer fuer Vollsicherung alle 2 Minuten

[Timer]
OnCalendar=*:0/2
Persistent=true
Unit=backup-full.service

[Install]
WantedBy=timers.target
```

NOTE

Heißen sowohl Service Unit als auch Timer Unit gleich, so kann auf die Eigenschaft **Unit** verzichtet werden!

Die Einstellung `OnCalendar=*:0/2` legt fest, dass der Timer alle 2 Minuten ausgelöst wird. `Persistent=true` sorgt dafür, dass der Timer auch dann ausgelöst wird, wenn das System zum Zeitpunkt des geplanten Auslösens ausgeschaltet war. In diesem Fall wird der Timer beim nächsten Systemstart sofort ausgelöst, um die verpasste Ausführung nachzuholen.

Verifizierung von Service und Timer Unit

Um sicherzustellen, dass keine syntaktischen Fehler innerhalb der Service und Timer Unit vorhanden sind, hat man die Möglichkeit den nachfolgenden Befehl auszuführen:

```
systemd-analyze verify backup-full.service backup-full.timer
```

TERMINAL

Aktivierung und Überwachung des Services

Der Job kann manuell gestartet werden mit:

```
systemctl start backup-full.timer
```

TERMINAL

Er kann aber auch dauerhaft aktiviert werden mit:

```
systemctl enable backup-full.timer
```

TERMINAL

Analog kann jetzt zum Beispiel auch der Cron für ein inkrementelles Backup umgeschrieben werden!

Weitere Beispiele

Bearbeiten von `/etc/resolv.conf`

Erstellung eines Skripts für die Eintragung von Nameservern in die Datei `/etc/resolv.conf` :

```
#!/bin/bash
```

TERMINAL

```
cat >/etc/resolv.conf <<'EOF'
domain <Domain>
search <Domain>
nameserver <IP>
...
EOF
```

Im Anschluss kann eine entsprechende Service Unit erstellt werden, die das Skript auch ausführt:

```
[Unit]
Description=Write static /etc/resolv.conf
After=network.target
Wants=network.target

[Service]
Type=oneshot
ExecStart=/usr/local/sbin/write_resolv.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
```

TERMINAL